

Омельяненко Кирило Андрійович

ПЗПІ-25-6

Лабораторна робота №4

1. Тема та мета лабораторної роботи

Тема: Проведення Code Review, рефакторинг програмного коду та оцінка його якості за допомогою інструментів статичного аналізу.

Мета роботи: набути практичних навичок проведення Code Review, виявлення проблем якості програмного коду, виконання рефакторингу без зміни функціональності програми, використання інструментів статичного аналізу коду (SonarLint) та регресійного тестування для перевірки коректності внесених змін.

2. Результати Code Review одnogрупника

№	Рядок коду	Проблема	Категорія	Рекомендація
1	17	Використання значення 2 без пояснення призначення	Magic Number	Винести значення в константу <code>RATING_MULTIPLIER</code>
2	8–10	Використання публічних полів замість властивостей	Encapsulation Violation	Замінити поля на властивості <code>get/set</code>
3	27–34	Використання магічних чисел 10 та 20	Magic Number	Винести значення в окремі константи
4	46	Конкатенація рядків через оператор <code>+</code>	Readability	Використати інтерполяцію рядків
5	52–55	Відсутня явна обробка <code>null</code> для колекції	Missing Validation	Використати <code>ArgumentNullException</code>

3. Посилання на PR або Issues

Посилання на результати Code Review:

<https://github.com/ruslankhyzhniakov-lang/LW3/issues>

4. Результати рефакторингу власного коду

Рефакторинг №1. Додавання валідації даних у конструктор Dish

БУЛО

```
public Dish(string name, string cuisine, decimal price)
{
    Name = name;
    Cuisine = cuisine;
    Price = price;
}
```

СТАЛО

```
public Dish(string name, string cuisine, decimal price)
{
    if (string.IsNullOrEmpty(name))
        throw new ArgumentException("Dish name cannot be empty.", nameof(name));

    if (string.IsNullOrEmpty(cuisine))
        throw new ArgumentException("Cuisine cannot be empty.", nameof(cuisine));

    if (price < 0)
        throw new ArgumentOutOfRangeException(nameof(price), "Price cannot be negative.");

    Name = name;
    Cuisine = cuisine;
    Price = price;
}
```

ЧОМУ

Було додано перевірку вхідних параметрів конструктора для запобігання створенню об'єктів із некоректними даними. Це підвищує надійність програми та спрощує пошук помилок.

Рефакторинг №2. Усунення магічних чисел

БУЛО

```
if (minRating < 0 || minRating > 5)
    throw new ArgumentException("Minimum rating must be between 0 and 5");
```

СТАЛО

```
public const int MinRating = 0;
public const int MaxRating = 5;

if (minRating < MinRating || minRating > MaxRating)
```

```
throw new ArgumentOutOfRangeException(  
    nameof(minRating),  
    "Minimum rating must be between 0 and 5.");
```

ЧОМУ

Магічні числа були винесені в окремі константи. Це покращує читабельність коду, спрощує його підтримку та дозволяє легко змінювати допустимий діапазон рейтингу в майбутньому.

Рефакторинг №3. Оптимізація методу GetRecommendedDishes

БУЛО

```
return dishes  
    .Where(d => CalculateAverageRating(reviews, d.Name) >= minRating)  
    .ToList();
```

СТАЛО

```
var averageRatings = reviews  
    .GroupBy(r => r.DishName, StringComparer.OrdinalIgnoreCase)  
    .ToDictionary(  
        group => group.Key,  
        group => Math.Round(group.Average(r => r.Rating), 2),  
        StringComparer.OrdinalIgnoreCase);  
  
return dishes  
    .Where(dish =>  
        averageRatings.TryGetValue(dish.Name, out var rating)  
        && rating >= minRating)  
    .ToList();
```

ЧОМУ

5. Звіт регресійного тестування

Попередня реалізація багаторазово обчислювала середній рейтинг для кожної страви, що призводило до зайвих витрат ресурсів. Після рефакторингу рейтинги обчислюються один раз і зберігаються у словнику (Dictionary), що підвищує продуктивність програми та зменшує кількість повторних операцій.

Після виконання рефакторингу було проведено регресійне тестування проєкту за допомогою фреймворку xUnit. Усі наявні тести були успішно виконані, що підтверджує відсутність порушень функціональності після внесення змін.

Результат тестування:

- Всього тестів: 11

- Успішно: 11
- Помилкок: 0

```

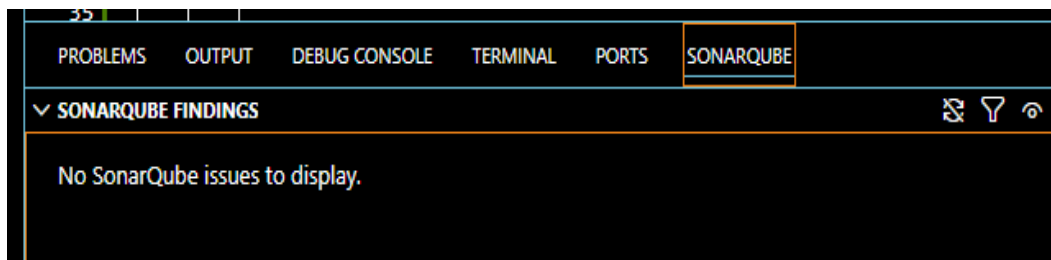
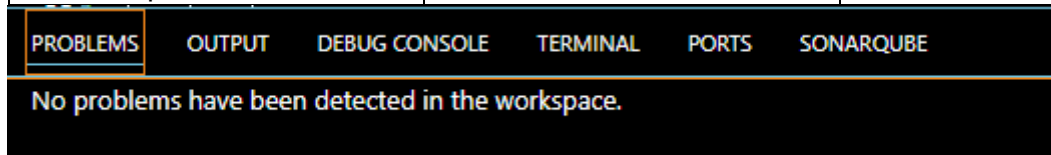
Восстановление завершено (0,3 с)
DishGuide net10.0 успешно выполнено (0,1 с) → DishGuide\bin\Debug\net10.0\DishGuide.dll
DishGuide.Tests net10.0 успешно выполнено (0,1 с) → DishGuide.Tests\bin\Debug\net10.0\DishGuide.Tests.dll
[xUnit.net 00:00:00.00] xUnit.net VSTest Adapter v3.1.4+50e68bbb8b (64-bit .NET 10.0.8)
[xUnit.net 00:00:00.30] Discovering: DishGuide.Tests
[xUnit.net 00:00:00.33] Discovered: DishGuide.Tests
[xUnit.net 00:00:00.34] Starting: DishGuide.Tests
[xUnit.net 00:00:00.52] Finished: DishGuide.Tests
DishGuide.Tests (тест) net10.0 успешно выполнено (1,4 с)

Сводка теста: всего: 11; сбой: 0; успешно: 11; пропущено: 0; длительность: 1,3 с
Сборка успешно выполнено через 2,3 с

```

6. Порівняння метрик «ДО / ПІСЛЯ»

Метрика	До рефакторингу	Після рефакторингу
SonarLint Issues	0	0
Warnings (Problems)	0	0
Цикломатична складність	Вища	Нижча
Магічні числа	Присутні	Усунено
Повторні обчислення рейтингів	Присутні	Усунено



Після виконання рефакторингу було повторно проведено аналіз коду за допомогою SonarLint. Нових проблем якості коду не виявлено. У процесі рефакторингу було усунуто магiчні числа, додано валідацію вхідних даних та оптимізовано алгоритм формування рекомендацій. Це позитивно вплинуло на підтримуваність і продуктивність програмного забезпечення.

7. Підсумкова рефлексія

Під час виконання лабораторної роботи я отримав практичний досвід проведення Code Review та аналізу якості програмного коду. Найбільш корисним було виявлення проблем у коді як одногрупника, так і у власному проєкті, оскільки це дозволило краще зрозуміти принципи написання якісного програмного забезпечення. У процесі рефакторингу я навчився додавати валідацію даних, усувати магічні числа та покращувати продуктивність алгоритмів. Також було корисно використовувати SonarLint для перевірки якості коду та виконувати регресійне тестування після внесення змін. Отримані знання допоможуть мені в подальшій розробці програмних проєктів та підтримці їхньої якості.